[승코의 차근차근 Flutter] 07, Provider + Database(Sqflite)

깃허브 링크 ⬇️

해당 코드 보러가기(깃허브)

Flutter로 To-Do 앱 만들기: 초보자를 위한 Provider와 Sqflite 완벽 가이드

Flutter로 간단한 To-Do 앱을 만드는 방법을 초보자도 쉽게 이해할 수 있도록 설명합니다. 특히 **Provider**를 사용한 상태 관리와 **Sqflite**를 활용한 로컬 데이터베이스 저장에 초점을 맞춰, 두 기술의 개념을 폭넓고 쉽게 풀어보고, 이들이 앱에서 어떻게 동작하는지 코드와 함께 살펴보겠습니다.

To-Do 앱 소개

이 To-Do 앱은 사용자가 할 일을 관리할 수 있는 간단한 앱입니다. 주요 기능은 다음과 같습니다:

- 할 일 목록 보기
- 새로운 할 일 추가
- 기존 할 일 수정
- 할 일 완료 체크
- 할 일을 길게 눌러 삭제

이 앱은 **Provider**를 통해 앱의 상태(할 일 목록)를 관리하고, **Sqflite**를 사용해 데이터를 휴대폰에 영구적으로 저장합니다.

---

1. Provider: 앱의 상태를 쉽게 관리하기

Provider란?

Provider는 Flutter 앱에서 **상태 관리**(State Management)를 쉽게 해주는 도구입니다. 상태 관리란 앱의 데이터를 관리하고, 데이터가 바뀌면 화면(UI)을 자동으로 갱신하는 것을 말합니다. 예를 들어, To-Do 앱에서 새로운 할 일을 추가하면 목록이 화면에 바로 반영되어야 합니다. Provider는 이런 작업을 간단하게 만들어줍니다.

비유로 설명하자면, Provider는 **가게의 점원** 같은 역할을 합니다:

- **가게 창고** (= 데이터): 할 일 목록 같은 데이터를 저장합니다.
- **점원** (= Provider): 창고에서 데이터를 꺼내 손님(= 화면)에게 보여주고, 손님이 요청하면 창고를 갱신합니다.
- **손님** (= UI): 점원에게 데이터를 요청해 화면에 표시합니다.

Provider는 앱 전체에서 데이터를 공유하고, 데이터가 바뀌면 필요한 화면 부분만 새로 고칩니다.

Provider의 핵심 개념

1. **ChangeNotifier**: 상태가 바뀌었음을 알리는 클래스입니다. 데이터가 변경되면 `notifyListeners()`를 호출해 화면에 "데이터가 바뀌었다!"고 알려줍니다.
2. **ChangeNotifierProvider**: 앱 전체에 데이터를 제공하는 역할을 합니다. 앱 시작 시 `TodoProvider`를 설정해 모든 화면에서 데이터를 공유할 수 있습니다.

3. **Consumer**: 데이터를 사용하는 화면 부분입니다. `Consumer`는 Provider가 제공하는 데이터를 듣고(lisiten), 데이터가 바뀌면 화면을 새로 그립니다.

To-Do 앱에서 Provider의 역할

`TodoProvider` 클래스는 앱의 할 일 목록(`List<Todo>`)을 관리합니다. 주요 메서드는 다음과 같습니다:

- **loadTodos()**: 데이터베이스에서 할 일 목록을 가져와 `_todos` 리스트에 저장하고, `notifyListeners()`를 호출해 화면을 갱신.
- **addTodo(Todo todo)**: 새로운 할 일을 데이터베이스에 추가한 뒤 목록을 갱신.
- **updateTodo(Todo todo)**: 할 일을 수정하고 목록을 갱신.
- **removeTodo(int id)**: 할 일을 삭제하고 목록을 갱신.

```
class TodoProvider with ChangeNotifier {
  List<Todo> _todos = [];

  List<Todo> get todos => _todos;

  Future<void> loadTodos() async {
    _todos = await DatabaseHelper().getTodos();
    notifyListeners();
  }

  Future<void> addTodo(Todo todo) async {
    await DatabaseHelper().insertTodo(todo);
    await loadTodos();
  }
  // ... (updateTodo, removeTodo 메서드)
}
```

위 코드에서 `notifyListeners()`는 "할 일 목록이 바뀌었으니 화면을 새로 그려!"라고 UI에 신호를 보내는 역할을 합니다. 예를 들어, 새로운 할 일을 추가하면 `addTodo`가 호출되고, `loadTodos`가 최신 데이터를 가져오며, `notifyListeners()`가 화면을 갱신합니다.

Provider 사용의 장점

- **쉽다**: 복잡한 코드를 작성하지 않아도 상태를 관리할 수 있습니다.
- **효율적이다**: 필요한 화면 부분만 갱신해 앱이 빠르게 동작합니다.
- **재사용 가능**: 앱 어디서나 `Provider.of<TodoProvider>(context)`로 데이터에 접근할 수 있습니다.

---

2. Sqflite: 휴대폰에 데이터를 저장하기

Sqflite란?

Sqflite는 Flutter에서 **SQLite**라는 데이터베이스를 사용하는 도구입니다. SQLite는 휴대폰에 데이터를 영구적으로 저장하는 작은 데이터베이스입니다. 예를 들어, To-Do 앱에서 할 일을 추가하면 앱을 껐다 켜도 그 데이터가 남아 있어야 하죠. Sqflite는 이런 데이터를 저장하고 관리하는 역할을 합니다.

비유로 설명하자면, Sqflite는 **노트북** 같은 역할을 합니다:

- **노트북** (= 데이터베이스): 할 일 목록을 적어둡니다.
- **필기** (= 데이터 저장): 새로운 할 일을 적거나, 기존 할 일을 수정/삭제합니다.
- **읽기** (= 데이터 조회): 노트북을 펴서 할 일 목록을 확인합니다.

Sqflite는 앱이 종료되어도 데이터를 잃지 않도록 저장하고, 필요할 때 데이터를 꺼내옵니다.

Sqflite의 핵심 개념

1. **데이터베이스**: 데이터를 저장하는 파일(예: `todo_database.db`)입니다.
2. **테이블**: 데이터를 정리하는 표입니다. 이 앱에서는 `todos`라는 테이블에 할 일을 저장합니다.
3. **CRUD 작업**:
    - **Create** (생성): 새로운 할 일 추가.
    - **Read** (읽기): 할 일 목록 가져오기.
    - **Update** (갱신): 할 일 수정.
    - **Delete** (삭제): 할 일 삭제.
4. **싱글톤 패턴**: 앱에서 데이터베이스를 하나만 사용하도록 보장합니다.

To-Do 앱에서 Sqflite의 역할

`DatabaseHelper` 클래스는 Sqflite를 사용해 데이터베이스를 관리합니다. 이 클래스는 **싱글톤 패턴**을 사용해 앱 전체에서 하나의 데이터베이스만 사용하도록 합니다. 주요 메서드는 다음과 같습니다:

- **_initDatabase()**: `todo_database.db` 파일을 생성하거나 열고, `todos` 테이블을 만듭니다.
- **getTodos()**: `todos` 테이블에서 모든 할 일을 가져옵니다.
- **insertTodo(Todo todo)**: 새로운 할 일을 테이블에 추가.
- **updateTodo(Todo todo)**: 기존 할 일을 수정.
- **deleteTodo(int id)**: 할 일을 삭제.

```
class DatabaseHelper {
  static final DatabaseHelper _instance = DatabaseHelper._internal();
  factory DatabaseHelper() => _instance;
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    String path = join(await getDatabasesPath(), 'todo_database.db');
    return await openDatabase(path, version: 1, onCreate: _onCreate);
  }

  Future _onCreate(Database db, int version) async {
    await db.execute('''
      CREATE TABLE todos(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT,
        isDone INTEGER
      )
    ''');
```

```
  }
}
```

위 코드에서 `todos` 테이블은 `id`(고유 번호), `title`(할 일 제목), `isDone`(완료 여부)을 저장합니다. `id`는 자동으로 증가하는 번호로, 각 할 일을 구분합니다. `isDone`은 0(미완료) 또는 1(완료)로 저장됩니다.

Sqflite 사용의 장점

- **데이터 유지**: 앱을 껐다 켜도 할 일 목록이 사라지지 않습니다.
- **간단한 사용**: 복잡한 서버 없이도 데이터를 저장하고 관리할 수 있습니다.
- **효율적**: SQLite는 가벼운 Lightweight이라 모바일 앱에 적합합니다.

---

3. Provider와 Sqflite의 팀워크

Provider와 Sqflite는 서로 다른 역할을 하지만, 함께 사용하면 강력한 조합이 됩니다:

- **Provider**: 할 일 목록을 메모리에 보관하고, 화면에 데이터를 보여줍니다.
- **Sqflite**: 할 일 목록을 휴대폰에 영구적으로 저장합니다.
- **연결**: `TodoProvider`가 `DatabaseHelper`를 호출해 데이터를 저장/가져오고, 상태를 갱신하면 UI가 새로 그려집니다.

예를 들어, 사용자가 새로운 할 일을 추가하면:

1. `TodoListScreen`에서 "Add" 버튼을 누르면 입력한 제목으로 `Todo` 객체를 만듭니다.
2. `TodoProvider`의 `addTodo` 메서드가 호출되어 `DatabaseHelper`의 `insertTodo`를 통해 데이터를 저장.
3. `loadTodos`가 호출되어 최신 데이터를 가져오고, `notifyListeners()`로 UI를 갱신.
4. `Consumer`가 이를 감지해 화면에 새로운 할 일 목록을 표시.

---

4. 프로젝트 구조와 주요 코드 설명

Todo 모델

`Todo` 클래스는 할 일 데이터를 정의합니다. `id`, `title`, `isDone` 필드를 가지며, 데이터베이스와 객체 간 변환을 위해 `fromMap`과 `toMap` 메서드를 제공합니다.

```
class Todo {
  int? id;
  String title;
  bool isDone;

  Todo({this.id, required this.title, this.isDone = false});

  factory Todo.fromMap(Map<String, dynamic> json) ⇒
      Todo(id: json["id"], title: json["title"], isDone: json["isDone"] == 1);

  Map<String, dynamic> toMap() {
    final map = {"title": title, "isDone": isDone ? 1 : 0};
    if (id != null) map["id"] = id!;
    return map;
  }
}
```

UI 구성

- **TodoListScreen**: 할 일 목록을 표시하고, 추가/수정/삭제를 처리합니다. `FutureBuilder`로 데이터 로딩을 관리하고, `Consumer`로 상태 변화를 반영.
- **EditTodoScreen**: 기존 할 일을 수정하는 화면. `TextEditingController`로 입력을 받고, `TodoProvider`를 호출해 갱신.
- **MyApp**: 앱의 시작점으로, `ChangeNotifierProvider`를 사용해 `TodoProvider`를 앱 전체에 제공.

---

5. 초보자를 위한 팁

- **Provider 시작하기**: `ChangeNotifierProvider`를 앱의 최상단(`MyApp`)에 설정하면 모든 화면에서 데이터를 공유할 수 있습니다.
- **Sqflite 시작하기**: 데이터베이스는 처음에 설정이 조금 복잡하지만, `DatabaseHelper`처럼 싱글톤 패턴을 사용하면 관리하기 쉽습니다.

- **디버깅**: 데이터가 화면에 안 보인다면, `notifyListeners()`가 호출되었는지, 데이터베이스 작업이 성공했는지 확인하세요.
- **확장 아이디어**: 할 일에 마감일, 우선순위, 카테고리를 추가해 앱을 더 멋지게 만들어보세요!

---

6. 결론

이 To-Do 앱은 **Provider**와 **Sqflite**를 사용해 Flutter에서 상태 관리와 데이터 저장을 구현한 훌륭한 사례입니다. Provider는 상태를 관리하고 UI를 갱신하는 일을 쉽게 만들어주고, Sqflite는 데이터를 영구적으로 저장해 앱의 신뢰성을 높입니다. 이 두 기술을 이해하면 더 복잡한 앱도 만들 수 있는 기반이 됩니다.